controlp.io — Cowork Rebuild Prompt

Project: controlp.io
Owner: Jeremy Waters (Qallus LLC) — jwaters@qallus.co
Scope: Full rebuild as a production Next.js ecommerce + operations platform
Build mode: Staged, one-page-at-a-time with explicit gating

READ THIS FIRST — Non-negotiable rules

1. Read all five mockups before writing any code.

The mockups in this folder are the authoritative design source. Do not invent UI. Do not pull patterns from elsewhere. Match what's in the mockups. Read them in this order:

1. `controlp-home.html` — marketing homepage, nav, hero, categories, product grid, Tesla wraps section, reviews, SMS CTA
2. `controlp-shop.html` — collection/browse page with sidebar filters, sort, pagination
3. `controlp-product.html` — product detail with image gallery + configurator + tabs + reviews
4. `controlp-customer-dashboard.html` — logged-in customer account with orders, drafts, artwork library
5. `controlp-admin.html` — dark-themed super-admin dashboard with KPIs, orders table, production queue, vendor sync

2. Tech stack is locked. Do not substitute.

Layer	Choice
Framework	Next.js 16.1.6 (App Router)
Runtime	React 19
Language	TypeScript (strict)
Styling	Tailwind v4 + Shadcn UI primitives
UI primitives	Radix (via Shadcn), Lucide React icons
Fonts	Inter (body) + Geist (display, -0.02em tracking)
DB/Auth	Supabase (self-hosted at supabase.cueallus.com) + NextAuth v5
Payments	Stripe (Checkout + webhooks)
Comms	Twilio (SMS), Nodemailer (transactional email, Hostinger SMTP), Resend (marketing)
File storage	Supabase Storage (artwork files, proofs)
Vendor APIs	4Over (https://4over.com), B2Sign (https://www.b2sign.com)
Deploy	Coolify on Hostinger VPS

Do not add libraries without asking. No Framer Motion, no Zustand, no TanStack Query, no React Hook Form without explicit approval. Use built-in Next.js + React primitives first.

3. One page at a time. Build → verify → confirm → move on. No batch-building multiple routes before checking in.

4. Self-check step. Before writing code in each phase, produce a short written summary of (a) what you're about to build, (b) which mockup it's based on, (c) what data it touches, (d) what you'll do if stuck. I read that summary before giving you the go.

Design System (lifted from mockups)

Light theme (storefront + customer dashboard):

```
css
```

```
:root {
  --background: 0 0% 100%;
  --foreground: 240 10% 3.9%;
  --card: 0 0% 100%;
  --muted: 240 4.8% 95.9%;
  --muted-foreground: 240 3.8% 46.1%;
  --border: 240 5.9% 90%;
  --primary: 240 5.9% 10%;
  --primary-foreground: 0 0% 98%;
  --radius: 0.625rem;
}
```

Dark theme (super-admin only):

```css
:root {
  --bg: #0a0d12;
  --panel: #11161d;
  --panel-2: #151b23;
  --border: #1f2833;
  --text: #e6edf5;
  --text-dim: #9aa7b8;
  --accent: #3b82f6;
}
```

Icons: Lucide React only. No emojis anywhere in the UI. Match the stroke-width and sizing used in the mockups (icon 16px, icon-sm 14px, icon-lg 20px, icon-display 64–112px for product placeholders).

PHASE 1 — Foundation

1. Repo setup

- Initialize Next.js 16.1.6 App Router with TypeScript, Tailwind v4, ESLint
- Install Shadcn CLI, initialize with the HSL token set above
- Install lucide-react, @supabase/ssr, @supabase/supabase-js, next-auth@beta, stripe, twilio, nodemailer, resend, zod
- Add `fonts.ts` loading Inter + Geist from `next/font/google`, apply Geist with `--font-display` CSS variable and `.font-display` utility class with `letter-spacing: -0.02em`

2. Supabase schema (v1)

Tables (all with RLS enabled):

- `users` (extends auth.users) — role enum: customer | staff | admin
- `addresses` — billing + shipping, linked to user
- `products` — sku, slug, category, base_price, vendor (in_house | 4over | b2sign), active
- `product_variants` — size/material/finish combinations with price deltas
- `product_options` — grommets, hemming, pole_pockets etc. with pricing rules
- `orders` — status enum: new | proof_pending | proof_approved | in_production | shipped | delivered | cancelled | refunded
- `order_items` — product_id, variant_id, options (jsonb), qty, unit_price, artwork_file_id
- `artwork_files` — supabase storage ref, user_id, filename, mime, size, thumbnail_url
- `design_drafts` — in-progress configurator state saved by customer
- `proofs` — order_item_id, url, customer_approved_at, revisions (jsonb array)
- `shipments` — carrier, tracking_number, status, shipped_at, delivered_at
- `vendor_jobs` — order_item_id, vendor, vendor_order_id, vendor_status, sync_state, last_synced_at
- `sms_messages`, `email_messages` — outbound communication log
- `coupons`, `store_credits`

Generate TypeScript types with `supabase gen types`. Commit to `lib/database.types.ts`.

3. Auth foundation

- NextAuth v5 with Supabase adapter
- Providers: Email (magic link via Resend), Google OAuth
- `/auth/signin`, `/auth/register`, `/auth/callback` routes
- Middleware protecting `/account/*` (customer) and `/admin/*` (staff + admin)
- Session includes user role — checked in middleware

4. Shared components

Build these Shadcn-based atoms into `components/ui/`:

- Button (primary, outline, ghost, sizes)
- Card

- Badge (status variants: new, proof_pending, in_production, shipped, delivered)
- Input, Select, Checkbox, Textarea
- Dropdown menu (hover + click)
- Table
- Tabs

Build these app-specific into `components/`:

- `<SiteHeader />` with sticky nav, search (⌘K), cart badge, user menu
- `<SiteFooter />`
- `<AnnouncementBar />`
- `<ProductCard />` — matches homepage/shop grid cards exactly
- `<ProductImagePlaceholder />` — the line-icon-on-gradient placeholder used throughout
- `<RatingStars />` — filled + empty SVG stars
- `<EmptyState />`

5. File structure

```
app/
  (marketing)/
    page.tsx                    # homepage
    shop/
      page.tsx                  # collection
      [category]/page.tsx
    products/[slug]/page.tsx    # product detail
    about/, contact/, blog/
  (auth)/
    signin/, register/
  account/
    layout.tsx                  # customer sidebar
    page.tsx                    # overview
    orders/, drafts/, artwork/, addresses/, settings/
  admin/
    layout.tsx                  # dark theme shell
    page.tsx                    # admin dashboard
    orders/, production/, customers/, products/, vendors/, analytics/
  api/
    webhooks/stripe/, webhooks/4over/, webhooks/b2sign/, webhooks/twilio/
    trpc or route handlers as needed
```

```
lib/
  supabase/, auth/, pricing/, vendors/4over.ts, vendors/b2sign.ts
```

GATE 1: Before Phase 2, show me the schema migration files, the route tree, and the bare `<SiteHeader />` running locally. I confirm. Only then proceed.

PHASE 2 — Storefront + Customer Account

6. Homepage (`app/(marketing)/page.tsx`)

Mirror `controlp-home.html` exactly. Sections in order:

- Announcement bar
- Sticky header with Shop mega-menu (hover-triggered)
- Hero with badge, split headline, floating stat cards
- Vendor trust marquee (3M, Avery, 4Over, B2Sign, Max Pro, Rowmark, SAi Flexi)
- Shop by category (8 tiles)
- Popular products grid with filter tabs
- Tesla wraps dark section (5 model cards)
- How it works (3 steps)
- Business cards feature strip
- Customer reviews (3 testimonials)
- SMS newsletter CTA
- Footer

Products and categories are loaded from Supabase. Tesla model cards link to `/shop/vehicle-wraps?model=<slug>`.

7. Shop/Collection (`app/(marketing)/shop/page.tsx`)

Mirror `controlp-shop.html`. Requirements:

- URL state for filters (category chips, material, finish, availability, turnaround, price range, rating) — use `useSearchParams`
- Server-rendered initial list (SEO), client hydration for filter interactions
- Sort dropdown changes URL param and re-fetches
- Pagination via URL param (`?page=2`)

- Sidebar filters collapse on mobile into a sheet

8. Product Detail + Configurator (`app/(marketing)/products/[slug]/page.tsx`)

Mirror `controlp-product.html`. Core work:

- Image gallery with 5 thumbnails (one is a video placeholder)
- Material swatch strip
- **Configurator engine** — the critical piece:
  - Size picker (presets + custom W×H inputs)
  - Material selector (with per-option price deltas)
  - Finish, Sides, Grommets, Finishing checkboxes (hemming, wind slits, pole pockets, rope reinforcement)
  - Quantity stepper
  - Turnaround selector (standard/rush/same-day with fee modifiers)
  - Artwork upload (dropzone → Supabase Storage) or "Upload later" / "I need design help"
- **Live price calculation** on every option change:
  - `price = (base_sqft_rate × sqft × material_multiplier × side_multiplier) + fixed_option_addons`
  - Abstract into `lib/pricing/calculate.ts` taking `(product, variant, options, qty)` → `{ subtotal, breakdown[] }`
- Sticky price + CTA card on desktop
- "Estimated ships by" date derived from turnaround choice + current time
- Save-as-draft creates a `design_drafts` row tied to user (or anonymous localStorage if logged out)
- Tabs: Description, Specifications, Shipping & Returns, Reviews

9. Cart (`app/cart/page.tsx`)

- Line items with image, variant summary, options list, qty adjuster, remove
- Coupon input
- Subtotal, shipping (free if $75+), tax estimate (Arizona), total
- Guest or logged-in checkout options

10. Checkout

- Stripe Checkout for payment (hosted, not embedded — simpler, more secure)
- On success webhook: create order rows, dispatch `vendor_jobs` for vendor-fulfilled items, trigger order-confirmation email + SMS
- Order confirmation page with order number, timeline preview, artwork upload reminder if not attached

11. Customer auth + `/account` layout

Match `controlp-customer-dashboard.html` sidebar nav.

12. Account → Overview

4 stat cards (Total orders, Active orders, Saved designs, Store credit) + Active orders (with progress bars), Recent orders table, Design drafts list, Artwork library grid, Quick reorder.

13. Account → Orders

Full order history with filters, order detail pages with timeline, proof download, tracking link, reorder button.

14. Account → Design Drafts

List of saved configurator states, click to resume (loads product detail page with saved options preselected).

15. Account → Artwork library + Addresses + Payments

- Artwork: grid view, preview modal, tag/delete
- Addresses: CRUD with default flags
- Payment methods: Stripe-hosted management portal link (no storing card data)

GATE 2: Ship phase 2 to staging. I test end-to-end: add product → configure → cart → checkout → order confirmation. Only then proceed.

PHASE 3 — Admin / Super-Admin

16. Admin layout (`app/admin/layout.tsx`)

Dark theme using the CSS variables defined above. Sidebar nav matches `controlp-admin.html`:

- Operations: Dashboard, Orders, Production, Shipments, Messages

- Catalog: Products, Vendors, Artwork
- Growth: Analytics, Customers, Marketing, Reviews
- System: Integrations, Settings

Role-gated to `staff` or `admin`.

17. Admin → Dashboard

Mirror `controlp-admin.html`:

- 4 KPI cards with sparklines (Revenue 7d, Orders 7d, In production, Approvals needed)
- Revenue line chart (this period vs previous, 14 days)
- Order pipeline bar stack
- Vendor sync status (4Over, B2Sign, Max Pro)
- Recent orders table with tabs (All/New/Flagged/Rush)
- Top products list
- Operational alerts feed
- New customers list
- Traffic sources bar chart
- Production queue with press utilization

18. Admin → Orders

- Full-featured data table (search, filter by status/vendor/date, bulk actions, CSV export)
- Order detail drawer/page: line items, customer, artwork, proof history, shipment tracking, internal notes, manual status override, refund
- Proof approval workflow: admin uploads proof PDF → customer gets SMS + email → customer clicks approve/request-changes

19. Admin → Production Queue

Kanban or sortable list showing jobs:

- Queued → On Press → Finishing → QC → Ready to Ship
- Drag to reorder priority
- Estimated completion time
- Press utilization %

20. Admin → Customers / Products / Vendors

- Customers: list + detail (order history, lifetime value, saved designs, notes, comms log)
- Products: CRUD for products, variants, options, pricing rules
- Vendors: 4Over and B2Sign sync dashboards (orders sent, status, errors, retry)

GATE 3: Demo admin dashboard end-to-end for me. Walk through a sample order from creation to delivery, including a vendor-fulfilled item. Only then proceed.

PHASE 4 — Integrations + Polish

21. 4Over integration (`lib/vendors/4over.ts`)

- Authenticate against 4over.com API
- Map internal product SKUs → 4Over SKUs (seed table)
- `submitOrder(orderItem)` — transforms our order into 4Over format, uploads artwork, returns vendor order ID
- `syncStatus(vendor_order_id)` — pulls current status, updates `vendor_jobs` row
- Background job (Vercel cron or Coolify cron) syncs statuses every 15 minutes
- Webhook endpoint if 4Over supports push notifications; otherwise fall back to polling

22. B2Sign integration (`lib/vendors/b2sign.ts`)

Same pattern as 4Over against b2sign.com API. Separate module so they can diverge freely.

23. Twilio SMS

- Order placed → customer SMS with order number + tracking link
- Proof ready → customer SMS with approval link
- Shipped → customer SMS with carrier + tracking
- Delivered → customer SMS with review request
- Inbound SMS handler (webhook) routes to admin Messages inbox
- Rate limit + opt-out handling (STOP keyword)

24. Email

- Transactional: order confirmation, proof ready, shipped, delivered, password reset

(Nodemailer + Hostinger SMTP)

- Marketing: newsletter, abandoned cart (Resend)
- React Email templates in `emails/` directory

25. Stripe webhooks

`checkout.session.completed`, `charge.refunded`, `invoice.paid` — handle idempotently with event ID dedup table.

26. Cron jobs

- Vendor status sync (every 15 min)
- Abandoned cart email (every hour for carts older than 4h)
- Daily summary email to admin (9am)
- Proof-pending-too-long alerts (every 2h)

27. Analytics

- PostHog or Plausible (self-hosted) for frontend analytics
- Custom admin analytics page: revenue by product, conversion funnel, vendor margin, customer LTV cohorts

28. Performance

- All images optimized via `next/image` with explicit width/height
- Product pages statically generated at build, ISR 1h
- Cart/account pages dynamic
- Lighthouse 95+ on all marketing pages

29. SEO

- Dynamic Open Graph images per product
- JSON-LD product schema with price, availability, rating
- Sitemap generation
- Robots.txt

30. Deployment

Deploy to Coolify VPS. Set up:

- Environment variables for all API keys
- Stripe webhook endpoint verified
- Domain: controlp.io (DNS already pointed)
- SSL via Let's Encrypt (Coolify handles)
- Staging subdomain: staging.controlp.io

GATE 4: Full production launch checklist:

- [ ] All 4 gates passed
- [ ] Stripe in live mode
- [ ] Vendor API keys live (not sandbox)
- [ ] Twilio phone number live, A2P 10DLC registered
- [ ] Transactional emails deliver (not in spam)
- [ ] Backups configured (Supabase daily, Coolify volume snapshots)
- [ ] Monitoring: Sentry for errors, Uptime Kuma for uptime
- [ ] Terms of Service + Privacy Policy live
- [ ] Cookie banner (if required by region)

Self-check format (every phase kickoff)

Before writing any code in a new phase, paste this back filled in:

```
PHASE: [number + name]
MOCKUP REFERENCE: [which .html file(s)]
WHAT I'M BUILDING: [1-2 sentence summary]
DATA TABLES TOUCHED: [list]
NEW DEPENDENCIES (if any): [must be approved]
OPEN QUESTIONS: [things you need clarified before starting]
DONE LOOKS LIKE: [the visible outcome I can verify]
```

Wait for my "go" before touching code.

Definition of Done

A page is done when:

1. It matches the mockup visually (spot check: spacing, typography, color tokens, icon styling)
2. It works on mobile + desktop (no horizontal scroll, interactive elements tap-sized)

3. Data flows through Supabase (no hardcoded mocks in the final build)
4. TypeScript has no errors, no `any`, no `@ts-ignore`
5. No console errors in browser
6. I've looked at it and said "ship it"